

Comparative Study on Defect Prediction Algorithms of Supervised Learning Software Based on Imbalanced Classification Data Sets

Jianxin Ge
School of Information Science and
Engineering
Yanshan University
Qin Huangdao, China
gejianxin3@outlook.com

Jiaomin Liu
School of Information Science and
Engineering
Yanshan University
Qin Huangdao, China

Wenyuan Liu
School of Information Science and
Engineering
Yanshan University
Qin Huangdao, China
wylu@vip.163.com

Abstract—With the development of high complexity and high integration of software systems, the quality of software has gradually received widespread attention in scientific research and engineering. Software defect prediction technology plays an important role in improving software quality, reducing software development time, and reducing testing expenses. It has also become one of the hot issues in the field of software engineering research in recent years. As an important learning method in machine learning, supervised learning is widely used in the classification and regression prediction with annotation data because of its high accuracy, mature theory, and simple calculation. However, the imbalanced classification problem of data sets is common in practical applications and seriously affects the performance of learning algorithm. This paper analyzes the characteristics of software forecasting technology from the perspective of supervised learning, and performs balance like processing on imbalanced classification NASA data sets (JM1, KC3, MC1). NASA data sets after application processing are used to conduct experiments on LWL, C4.5, Random forest, Bagging, Bayesian Belief Network, Multilayer Feed forward Neural Network, SVM and NB-K algorithms, and experimental data is analyzed and evaluated.

Keywords—software defect prediction, supervised learning, algorithm, data set, imbalanced classification

I. INTRODUCTION

Over the past few decades, the number of large-scale and highly-complex software systems has grown significantly. With the widespread use of big data and cloud computing platforms, software systems are now characterized by features such as synergy, autonomy, evolution, and polymorphism, etc. Moreover, there is a development trend of software platform network, system component, product family, development engineering, process standardization, production scale, and competition internationalization [1]. Industry data shows that in the past 10 years, the size and quantity of software systems and applications, even in the field of mobile terminals, have grown exponentially [2]. Due to the universal application of software systems in all walks of life, a large number of business processes and social work have shown a strong coupling to software systems, and the consequences of

software defects have become increasingly serious, especially it may cause serious consequences for the machine in the field of engineering that requires high reliability [3].

In recent years, with the rapid development of machine learning, supervised learning as an important part of machine learning has received widespread attention in the field of software defect prediction. Its algorithm application has also become a research hot spot. In view of this situation, this paper provides an important basis for the application and research of software defect prediction technology engineering through the practice of forecasting engineering technology and the analysis and evaluation of current mainstream algorithms.

II. SOFTWARE DEFECT PREDICTION TECHNOLOGY AND PREDICTION FRAMEWORK

This section mainly describes the basic concepts, software metrics, feature selection methods of software defect prediction, and proposes a simplified software defect prediction framework suitable for engineering applications.

A. Basic concepts of software defect prediction

Defects are often called bugs. The so-called software defect is a certain problem, error, or hidden functional defect that exists in computer software or program that damages normal operation ability. The existence of defects will cause software products unable to meet the needs of users to some extent. The original definition of defects in IEEE 729-1983 is: from the perspective of internal products, defects are problems in the process of software product development or maintenance, such as errors and faults; from the outside of the product, defects are a function failure or violation that the system needs to achieve [4]. In January 2017, IEEE issued ISO/IEC/IEEE 24765:2017 standard, which replaced the original IEEE 729-1983 standard to redefine software defects.

1. Disadvantages or deficiencies of substitutions in work products need to be repaired because they do not meet requirements and specifications; 2. Defects or deficiencies in the constituent elements of the project that need to be repaired or replaced due to non-compliance with requirements and

This work is supported by The Excellent Going Abroad Experts' Training Program in Hebei Province.

specifications; 3. Common terms can refer to faults (causes) and failures (impacts). [5]

In the process of software development, the production of software defects is unavoidable. There are many factors that cause software defects. The main reasons are: Human factors, Failure of communication, Unrealistic development time frames, Insufficient design, Low code implementation level, Insufficient version control, Defects from third-party programs or tools, Inadequate testing techniques.

B. Software Metrics Element

Software metrics is a continuous quantitative process of data definition, collection, and analysis of software development projects, processes, and their products. The goal is to understand, predict, evaluate, control, and improve it. Software metric element is an indicator or parameter that describes a software product or system, that is, a feature of the software [6]. At present, there are mainly process-oriented McCabe [7], Halstead [8], and object-oriented CK (Chidamber Kemmerer) [9] metrics.

C. Feature selection method

In machine learning, feature selection is a typical data preprocessing technique and plays a key role in classification tasks [10]. There are three main methods for combining feature selection with machine learning algorithms: Filter method, Wrapper method, and embedded method.

Filter method: By weighting the features of each dimension data, the importance of dimension features is represented by weights and then they are sorted according to the weights. The main methods are: Chi-squared test, information gain, Significance, and so on.

Wrapper method: The selection of the subset is considered as a search optimization problem, generate different combinations, evaluate the combinations, and compare them with other combinations. The main methods include recursive feature elimination algorithm and CfsSubsetEval.

Embedded method: Learn the best attributes for improving the accuracy of the model when the model is set. That is, in the process of determining the model, those attributes that are important to the training of the model are selected. Main methods are: Ridge Regression, LASSO (Minimal Absolute Compression) [11].

Because of the high computational efficiency of Filter method and the better prediction performance of Wrapper method under large-sample and multi-attribute datasets, some scholars have proposed a two-stage combined feature selection algorithm: Filter method is used as a preselector, and Wrapper method is used for further feature screening of the feature sets after preselector [12]. Since NASA dataset is small sample data, the number of attribute features is limited, and embedded method needs to determine the model. This article focuses on the difference evaluation and analysis of different learning algorithms. Therefore, this paper uses Filter method for feature selection.

III. DEFECT PREDICTION ALGORITHM BASED ON SUPERVISED LEARNING

Software defect prediction models mainly include: supervised learning-based predictive models, semi-supervised learning-based predictive models, unsupervised learning-based predictive models, regression-based predictive models, and attribute-reduction-based predictive models [13]. The main discussion is based on the supervisory learning prediction algorithm model. The following is a briefly introduction of C4.5, Randomforest, Bagging, Bayesian Belief Network, Multilayer Feedforward Neural Network, and SVM.

A. Instance-based prediction algorithm

Instance-based prediction method [14] first simply stores the training examples, then calculates the objective function based on local information of the sample, finds the sample most similar to the test sample, and gives the most similar result based on the local information. KNN algorithm [15] is an instance-based prediction algorithm that assumes that all instances correspond to points in the n -dimensional Euclidean space $\mathcal{R}^n$. The nearest neighbor of an instance is defined according to the standard Euclidean distance. That is, any instance x is represented as the following feature vector: $\langle a_1(x), a_2(x), \dots, a_n(x) \rangle$, where $a_r(x)$ represents the r th attribute value of the instance x . Then the distance between two instances x_i and x_j is defined as $d(x_i, x_j)$, shown in formula (1):

$$d(x_i, x_j) = \sqrt{\sum_{r=1}^n (a_r(x_i) - a_r(x_j))^2} \quad (1)$$

Given a software module test sample x , the discrete objective function $f: \mathcal{R}^n \rightarrow V$. Where V is the class of data set $\{v_1, \dots, v_s\}$, the training set is sampled with the nearest x , $x_1, \dots, x_k$, then the objective $\hat{f}(x_q)$, $\hat{f}(x_q)$ is the estimation of $f(x_q)$. For discrete values, according to the voting principle, it can be drawn

$$\hat{f}(x_q) \leftarrow \arg \max_{v \in V} \sum_{i=1}^k \delta(v, f(x_i)) \quad (2)$$

$\delta(v, f(x_i))$ is an indicator function, when $v = f(x_i)$, the function value is 1, and 0 otherwise. For the continuous function $\hat{f}(x_q)$, according to the calculated k nearest neighbors, there is

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^k f(x_i)}{k} \quad (3)$$

The different distance forms in KNN algorithm have a greater impact on instance-based learning algorithm. These distance forms include Euclidean Distance (ED), Manhattan distance (MAD), Chebyshev Distance (CD), Minkowski Distance (MID) and so on. European distance is more commonly used. However, the effect of KNN algorithm on imbalanced classification datasets is limited. Arvinder Kaur et al. [16] have achieved good results in applying local weighted learning algorithms (LWL) to software defect prediction. This paper introduces LWL algorithm into comparison experiment.

B. Decision tree based prediction algorithm

Decision tree is a tree structure (a binary tree or a non-binary tree). The basic decision tree algorithm has many improved versions of CART, CHAIR, ID3, C4.5, and so on. C4.5 algorithm [17] is a widely used algorithm in this series of algorithms. This algorithm is different from ID3 algorithm which uses information gain as a split criterion and uses information gain ratio as a criterion for tree node splitting. It effectively overcomes the bias of ID3 algorithm to multi-valued attributes. The main idea of information gain ratio is that if the decision tree is divided and the given samples can be classified, little information is needed. Let D be the division category of training tuples, then the entropy of D is expressed as:

$$info(D) = -\sum_{i=1}^n p_i \log_2(p_i) \quad (4)$$

Assuming that the training tuple D is divided by attribute A, the expected information for A to D division is:

$$info_A(D) = \sum_{j=1}^{|D|} \frac{|D_j|}{|D|} info(D_j) \quad (5)$$

The information gain (A) is the difference between the two:

$$gain(A) = info(D) - info_A(D) \quad (6)$$

The C4.5 algorithm defines "split information," which can be expressed as:

$$split_info_A(D) = -\sum_{j=1}^{|D|} \frac{|D_j|}{|D|} \log_2\left(\frac{|D_j|}{|D|}\right) \quad (7)$$

The gain rate defined by the C4.5 algorithm is:

$$gain_ratio(A) = \frac{gain(A)}{split_info_A(D)} \quad (8)$$

Random forest prediction algorithm is also a decision tree prediction algorithm. It consists of many decision trees because these decision trees are formed using random methods and are called random forests. There is no correlation between decision trees in random forests. When test data enters a random forest, it is actually to classify each decision tree to see which category this sample should belong to. Finally, the classification results are taken from all decision trees. The type of that is the final result (important consideration of the power of each tree comes in). Random forest can handle discrete values as well as continuous values. It has the advantage of high efficiency, high operating efficiency and accuracy in the classification of multidimensional feature data sets, and can generate a generalization error in classification process. The internal unbiased estimate generated during classification can detect the interaction between features and the degree of importance of the features in training process. Disadvantages may over fit in the case of large data noise.

C. Bayesian-based prediction algorithm

The Bayesian prediction model is a kind of prediction using Bayesian statistics. Bayesian statistics are different from general statistical methods, which not only use model information and data information, but also make full use of prior information. Thomas Bayes' statistical forecasting method is a time series forecast method based on dynamic model. When doing statistical inference, the general mode is: a priori information + overall distribution information + sample information $\rightarrow$ posterior distribution information.

In recent years, Rizky Tri Asmono [18] and others proposed a Naive Bayesian software defect prediction method based on absolute correlation weighting, and achieved good prediction results.

First calculate the likelihood weights. Since NASA data set is mostly digital in nature, it satisfies the normal. Then calculate the prior probability:

$$p(c) = \frac{\prod_{i=1}^n p(a_i|c)^{w_i}}{\prod_{i=1}^n p(a_i|c)^{w_i} + \prod_{i=1}^n p(a_i|\bar{c})^{w_i}} \quad (9)$$

Calculate class probability:

$$p(c|E) = p(c) \prod_{i=1}^n p(a_i|c) \quad (10)$$

Use this method to traverse all classes and use the highest probability class as the prediction result.

Bayesian Belief Network is also a prediction algorithm based on Bayesian prediction algorithm. It has a more flexible modeling method for class conditional probabilities $P(X|C)$ and does not require a given set of dataset properties. All conditions are independent, but it allows specifying which attribute conditions are independent. Bayesian network consists of two parts: network topology and probability table.

The advantages of Bayesian-based prediction algorithms are that they perform well for small-scale data, can handle multi-classification tasks, and are suitable for incremental training, especially when the amount of data exceeds memory. It is less sensitive to missing data and is often used for text classification. Disadvantages are: Compared with other classification methods, it has the smallest error rate. It needs to know the prior probability, and because the algorithm mechanism determines the classification by the a priori and the data to determine the posterior probability, so the classification decision has a certain error rate.

D. Prediction algorithm based on support vector machine

Support Vector Machine (SVM) was first proposed by Corinna Cortes and Vapnik et al. in 1995. It demonstrated many unique advantages in solving small-sample, non-linear and high-dimensional pattern recognition, and can be applied to function simulation and other machine learning problems. In machine learning, support vector machines are supervised learning models related to related learning algorithms that can analyze data, identify patterns, and be used for classification and regression prediction analysis.

P.A. Selvaraj and Dr.P. Thangaraj [19] proposed a software defect prediction model based on support vector machines and achieved good prediction results. In this prediction model, the penalty function will be used to verify that the classification surface has no segmentation error if the defect data set is optimally classified by hyperplane. At the same time, the closest data sample from the hyperplane has the largest distance from the hyperplane. The defect prediction is a two-class problem, ie, using training data to establish a classification function $d(x) = \pm 1$, the problem of finding the best classification surface is equivalent to solving the equation

(11) for optimal problem solving, where C is a penalty function.

$$\min_{w,b,\xi} = \frac{1}{2} w^T w + C \sum_{i \in T} \xi_i$$

$$\text{with constraints } l(y_i)(w^T \phi(y_i) + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0 \text{ for } i \in T, \quad (11)$$

Gayathri M, and A. Sudha [20] proposed a defect prediction model based on Multilayer Feedforward Neural Network and achieved good results. Therefore, this article also includes Multilayer Feedforward Neural Network in experimental comparison data.

E. Ensemble learning based prediction algorithm

The basic idea of ensemble learning prediction algorithm is to integrate several individual classifiers when classifying new instances, and to determine the final classification by taking a certain combination of the classification results of multiple classifiers to obtain better performance than a single classifier. If a single classifier is likened to a decision maker, the ensemble learning approach is equivalent to making multiple decisions among multiple decision makers. The more common integrated classifiers include: Bagging method [21], Boosting method [22] and Stack method [23].

The integrated classifier obtained by Bagging method is generally better than single classifier. The integrated classifier obtained by Boosting method may be significantly better than the classifier and single classifier obtained by Bagging method. However, traditional integration algorithm may suffer from the loss of some attribute information. Zhongbin Sun [24] proposed an improved new predictor that converts imbalanced binary classes into balanced multi-classes, and achieved good results.

F. Imbalanced classification data processing

The problem of imbalanced classification in real life is very common. For example, in fraud transaction prediction, the number of behaviors belonging to fraudulent transactions is generally much smaller than normal transactions. This situation is particularly evident in software defect prediction.

At present, the methods to solve the problem of classification imbalanced classification data are mainly divided into three categories [25]: a type of sampling adjustment method [26]. This method also achieves the effect of balancing data by adjusting the number of samples; the second type of cost-sensitive learning method [27], which can be used to optimize the performance of minority class classification by adjusting error classification cost; third type of ensemble learning method [28] is to solve classification imbalance problems by integrating multiple classification learning algorithms. In this paper, cost-sensitive learning is used to reset instance weights so that each class has the same total weight to deal with imbalanced problems.

G. Prediction algorithm evaluation index

Because the research content of this paper is software defect prediction, combined with NASA data set attribute

index, its essence is to predict whether there is a defect in the software module. Since the platform used in this paper is WEKA, the evaluation index is also based on WEKA platform confusion matrix. See TABLE II.

TABLE I. CONFUSION MATRIX OF RESULTS

Actual Results \ Prediction Results	Defect	No Defect
Defect	TP	FN
No Defect	FP	TN

^a. TP(True Positive), FP(False Positive), FN(False Negative), TN(True Negative).

According to TABLE II, confusion matrix can exist in four situations: the number of statistically defective modules is correctly classified, that is, the number of real cases, represented by TP; the number of correctly-classified non-defective modules is counted, that is, the number of true negative cases, represented by FP; Count the number of defective modules that have been incorrectly classified, that is, the number of false negatives, denoted by FN; and count the number of incorrectly classified modules that are incorrectly classified, that is, the number of false positives, which is represented by FP.

The prediction algorithm can be evaluated by Accuracy(A), Precision(P), Recall(R), F-Measure(FM), FP Rate(FPR), Matthews Correlation Coefficient (MCC) and ROC /PRC curves

IV. EXPERIMENTS AND ANALYSIS

This section compares the performance indicators of LWL, C4.5, Randomforest, Bagging, Bayesian Belief Network, Multilayer Feedforward Neural Network, and SVM algorithms, and analyzes the actual effects and adaptation fields of each algorithm. Select data set from NASA MDP library. Experimental platform is: JDK1.8.0_151, Weka3.8, MATLAB R2017b, IBM CPU: Intel Core i5 2.49GHz, 8GB of memory.

A. Introduction to experimental data sets

The template is designed so that author affiliations are not repeated each time for multiple authors of the same affiliation. Please keep your affiliations as succinct as possible (for example, do not differentiate among departments of the same organization). This template was designed for two affiliations.

The experiment considers three kinds of data sets of JM1, KC3, and MC1 in NASA MDP, taking into account factors such as program language diversity, imbalance ratio, and instance size. See TABLE III NASA DATASETS. In the table, "L" indicates program language; "F" indicates feature number; "I" indicates number of instances; "D" indicates the number of defective modules in the software; "ND" indicates the number of defective modules in the software; "DR" Indicates the defect rate in the software; "IR" indicates the classification imbalance rate. According to the study of Gray D [29] and others, there are problems such as data inconsistency and duplication in NASA's original data set. It

cleans NASA's original data set and provides a cleaned version that can be downloaded from Tera-PROMISE library. Experiments use cleaned version datasets to ensure data quality.

TABLE II. NASA DATASETS

Name	Datasets Attribute						
	<i>L</i>	<i>F</i>	<i>I</i>	<i>D</i>	<i>ND</i>	<i>DR</i>	<i>IR</i>
JM1	C	22	7720	1612	6108	20.9%	3.8
KC3	JAVA	40	194	36	158	18.6%	4.4
MC1	C/C++	39	1952	36	1916	1.8%	53.2

B. Experimental plan

Data preprocessing in this experiment mainly includes data cleaning, discrete discretization, dimensionality reduction, and imbalanced classification problems processing. Due to the use of NASA Cleaned data sets, the preliminary cleansing of the data set has been completed. This paper focuses on the differences in the performance metrics of different classification methods. Therefore, a unified feature selection method is used.

Select seven classification algorithms LWL, C4.5 (J48), Randomforest (RF), Bagging+ Randomforest (B), Bayesian Belief Network (BBN), Multilayer Feedforward Neural Network (MF), and SVM (SMO) to complete data classification training. In order to more effectively compare the performance of the above classification algorithms, the paper introduction of NB (Naïve Bayes) of Menzies T and others [30] is used as a comparison experiment. All experiments were conducted on Weka3.8 platform. There was no specific explanation for the selection of system default values. In order to ensure the reliability and accuracy of the experiment, all experiments were performed using 10 times 10-fold cross validation.

Experimental evaluation data indicators use the indicators described in section III.G of this paper.

C. Experimental results and analysis

Run experiments on Weka 3.8 platform. The experimental results of JM1 data set are shown in Table TABLE IV. The results of KC3 data set are shown in table TABLE V. The experimental results of MC1 data set are shown in table TABLE VI. The data in the tables retain four significant figures. The integrated learning Bagging classifier selects random forest method. For ease of display, "M" in the table indicates Metrics; "T" in the table indicates TP Rate; "F" in the table indicates FP Rate; "f" indicates F-Measure; "M" indicates MCC; "U" indicates AUC; "C" indicates PRC.

It can be seen from TABLE IV and Fig. 1 that there is no significant difference in the performance of each algorithm on the JM1 data set. Software defect prediction tends to pay more attention to whether defects can be fully predicted. That is, the relative accuracy index and recall are more noticeable. Among the eight classification algorithms implemented this time, BBN, LWL, and Bagging+ Randomforest (The classifier) algorithm has a high recall rate of 0.646, 0.643, and 0.641,

respectively. The AUC value reflects the classification ability of the algorithm. The Randomforest(RF), Bagging+Randomforest(classifier), and Bayesian Belief Network(BBN) algorithms perform well in terms of classification capabilities, reaching 0.710, 0.700, and 0.696, respectively, and their PRC values are also the best performance among the eight algorithms, 0.687, 0.701, and 0.682, respectively. Considering F-Measure value, Bagging+ Randomforest (classifier) is 0.630 and Bayesian Belief Network (BBN) 0.645 algorithm has similar classification ability, slightly better than 0.609 of Randomforest (RF) algorithm. The top three MCC scores in the fight against imbalance classification are Bagging+Randomforest (classifier), Bayesian Belief Network (BBN) and LWL, which are 0.300, 0.295 and 0.290, respectively, indicating that they have stronger imbalanced classification data adaptability.

TABLE III. EXPERIMENTAL RESULTS OF DIFFERENT DATA SETS(JM1)

M	Classification Algorithm							
	<i>LWL</i>	<i>J48</i>	<i>RF</i>	<i>B</i>	<i>BBN</i>	<i>MF</i>	<i>SMO</i>	<i>NB</i>
A	0.643	0.631	0.628	0.641	0.646	0.625	0.620	0.620
T	0.643	0.631	0.627	0.641	0.646	0.625	0.620	0.620
F	0.357	0.369	0.373	0.359	0.354	0.375	0.380	0.380
P	0.647	0.632	0.658	0.660	0.649	0.627	0.658	0.656
R	0.643	0.631	0.627	0.641	0.646	0.625	0.620	0.620
f	0.641	0.630	0.609	0.630	0.645	0.624	0.595	0.598
M	0.290	0.263	0.283	0.300	0.295	0.252	0.275	0.274
U	0.652	0.623	0.700	0.710	0.696	0.665	0.620	0.685
C	0.635	0.589	0.687	0.701	0.682	0.649	0.579	0.667

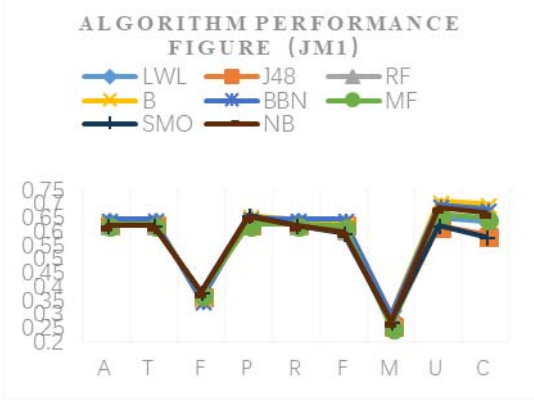


Fig. 1. Fig. 1. Algorithm Performance Figure(JM1)

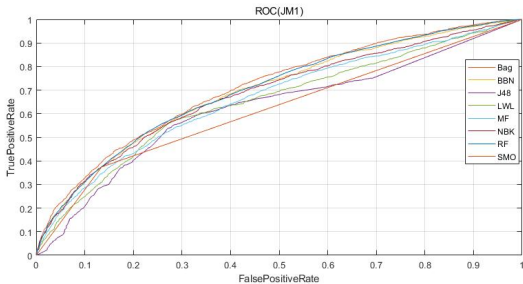


Fig. 2. Fig. 2. ROC Comparison Figure (JM1)

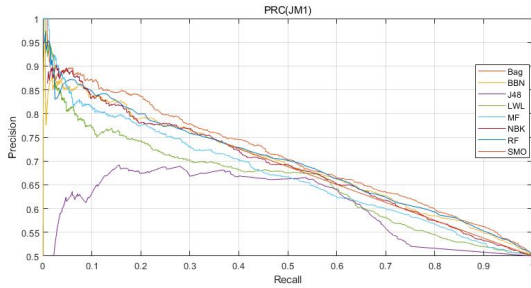


Fig. 3. Fig. 3. PRC Comparison Figure (JM1)

It can be seen from Fig. 2 ROC curve that Bagging+Randomforest(classifier), Randomforest(RF) and Bayesian Belief Network(BBN) algorithms perform well. Combined with Fig. 3.PRC curve, Bagging+ Randomforest(classifier) algorithm is obviously superior. Randomforest (RF) and Bayesian Belief Network (BBN) algorithms perform equally. Experimental results show that Bagging+Randomforest(classifier), Randomforest(RF), Bayesian Belief Network(BBN) and LWL algorithms perform better in JM1 data sets with more instances and fewer eigenvalues. Bagging+Randomforest(classifier) algorithm performs relatively excellent.

TABLE IV. EXPERIMENTAL RESULTS OF DIFFERENT DATA SETS(KC3)

M	Classification Algorithm							
	LWL	J48	RF	B	BBN	MF	SMO	NB
A	0.667	0.739	0.639	0.633	0.741	0.663	0.638	0.744
T	0.667	0.739	0.639	0.633	0.741	0.663	0.638	0.744
F	0.333	0.261	0.361	0.367	0.259	0.337	0.362	0.256
P	0.723	0.739	0.651	0.643	0.768	0.691	0.655	0.772
R	0.667	0.739	0.639	0.633	0.741	0.663	0.638	0.744
f	0.645	0.739	0.632	0.626	0.734	0.651	0.628	0.737
M	0.386	0.478	0.290	0.276	0.508	0.353	0.293	0.516
U	0.661	0.744	0.671	0.666	0.721	0.664	0.638	0.739
C	0.669	0.714	0.644	0.659	0.683	0.667	0.590	0.717

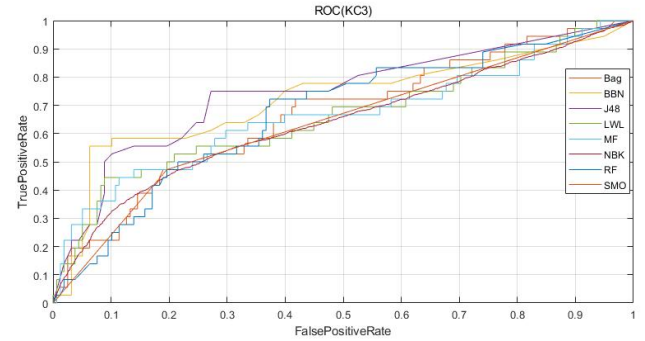


Fig. 5. Fig. 5.ROC Comparison Figure (KC3)

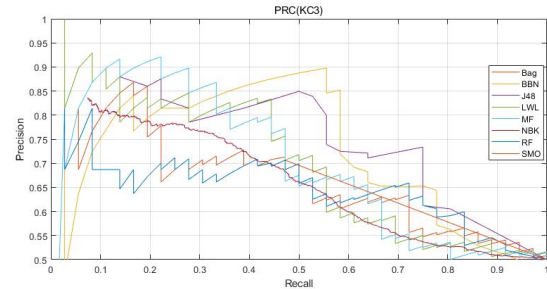


Fig. 6. Fig. 6.PRC Comparison Figure (KC3)

From ROC curve of Fig.5, it can be seen that NB-K, BBN and J48 algorithms all perform well, but in combination with Fig.6.PRC curve, BBN and J48 algorithms have a strong floating difference (saw-tooth), cannot guarantee monotonic consistency, so this round of experiments in NB-K algorithm has a clear advantage.

It can be seen from TABLE VI and Fig. 7 that the performance difference of each algorithm on MC1 data set is also obvious. Among the eight classification algorithms implemented this time, Bagging+Randomforest(classifier), BBN, and SMO algorithms all have high recall rates of 0.726, 0.718, and 0.716, respectively. In terms of AUC values, Bagging+Randomforest(classifier), Randomforest, and BBN performed well, reaching 0.840, 0.789, and 0.753, respectively, and SMO algorithm was 0.716. PRC value was also the best performing of the eight algorithms, which were 0.833, 0.759, 0.748 respectively. In combination with F-

ALGORITHM PERFORMANCE

FIGURE (KC3)

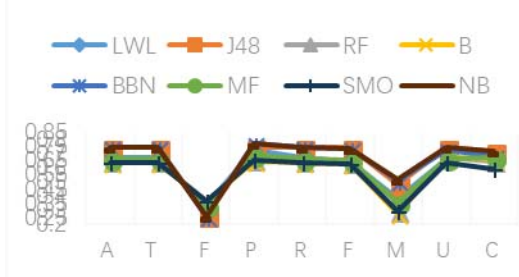


Fig. 4. Fig. 4.Algorithm Performance Figure (KC3)

It can be seen from TABLE V and Fig. 4 that the performance difference of each algorithm on KC3 data set is

Measure value, SMO algorithm has an optimal value of 0.716, Bagging+Randomforest (classifier) of 0.707, BBN of 0.704, and Randomforest of 0.656. The top three MCC scores in the fight against imbalance classification are Bagging+Randomforest(classifier), BBN, and Randomforest algorithms, which are 0.524, 0.483, and 0.465, respectively, indicating that they have stronger imbalanced classification data adaptability, but the MMC value of Bagging+Randomforest(classifier) algorithm is only 0.276 in KC3 data set, indicating that the algorithm has room for improvement in generalization ability.

TABLE V. EXPERIMENTAL RESULTS OF DIFFERENT DATA SETS(MC1)

M	Classification Algorithm							
	LWL	J48	RF	B	BBN	MF	SMO	NB
A	0.700	0.673	0.687	0.726	0.718	0.631	0.716	0.654
T	0.700	0.673	0.687	0.726	0.718	0.631	0.716	0.654
F	0.300	0.327	0.313	0.274	0.282	0.369	0.284	0.346
P	0.711	0.756	0.789	0.804	0.769	0.634	0.716	0.664
R	0.700	0.673	0.687	0.726	0.718	0.631	0.716	0.654
f	0.695	0.644	0.656	0.707	0.704	0.629	0.716	0.648
M	0.410	0.421	0.465	0.524	0.483	0.265	0.433	0.318
U	0.727	0.676	0.789	0.840	0.753	0.674	0.716	0.744
C	0.705	0.640	0.759	0.833	0.746	0.667	0.655	0.713

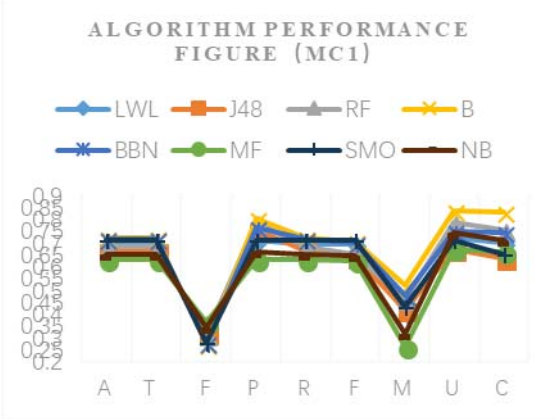


Fig. 7. Fig. 7. Algorithm Performance Figure (MC1)

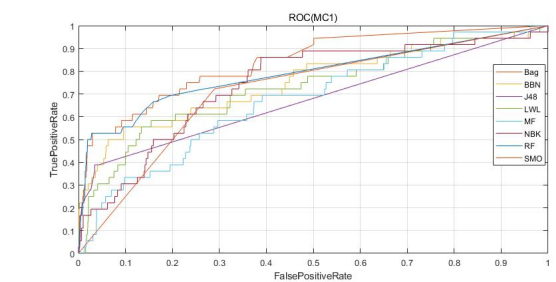


Fig. 8. Fig. 8. ROC Comparison Figure (MC1)

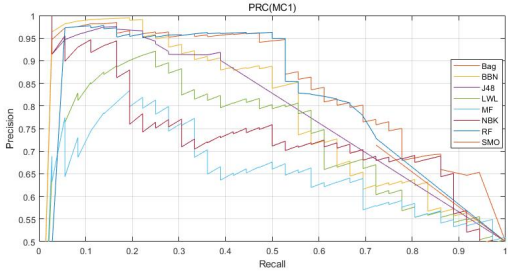


Fig. 9. Fig. 9. PRC Comparison Figure (MC1)

From Fig.8.ROC curve and Fig.9.PRC curve, it can be seen that Bagging+ Randomforest(classifier) and Randomforest algorithm perform well and have obvious advantages.

V. CONCLUSION

This paper presents a software defect prediction framework suitable for engineering applications through analysis and research of software defect prediction technology. Through the study of supervised learning software prediction algorithms, methods of solving imbalanced classification are analyzed. Instance-based prediction algorithm (LWL), decision tree-based prediction algorithm (C4.5, Randomforest), and ensemble-based learning prediction algorithm (Bagging+Randomforest (classifier)), Bayesian-based prediction algorithm (Bayesian Belief Network, NBK), vector machine-based prediction algorithm (SVM, Multi-layer Feedforward Neural Network) algorithms were used for simulation experiments and experimental results were analyzed. The results of the study show that there are differences in the performance of eight NASA datasets (JM1, KC3, and MC1), and ensemble-based learning algorithm Bagging+Randomforest (classifier) has strong capabilities in terms of classification capabilities. Randomforest algorithm based on decision trees and Les's BBN algorithm performed well; the advantages of Bayesian-based BBN and BN-K algorithm in imbalanced classification prediction are obvious; Bagging+ Randomforest (classifier) and BBN have good performance in terms of recall rate.

ACKNOWLEDGMENT

This work is supported by The Excellent Going Abroad Experts' Training Program in Hebei Province.

REFERENCES

[1] Yang Fuqing. On Software Engineering Technology Development [J]. Journal of Software, 2005, 16(01):1000-9825.

[2] Florian Hauer, Alexander Pretschner, Maximilian Schmitt and Markus Grötsch. The 28th International Symposium on Software Reliability Engineering (ISSRE): Industrial Evaluation of Search-Based Test Generation Techniques for Control Systems. October 23-26, 2017 [C]. Toulouse, France.c2017.

[3] Feng-Bin Sun; Jiliang Zhang. 2017 Annual Reliability and Maintainability Symposium (RAMS): Dealing with fractional failures and survivors in data analysis of reliability engineering. 23-26 Jan. 2017[C]. Orlando, FL, USA. C2017

[4] ANSI/IEEE Std 729-1983: IEEE Standard Glossary of Software Engineering Terminology.[S] IEEE, 1983.

- [5] ISO/IEC/IEEE Std 24765:2017(E).ISO/IEC/IEEE International Standard - Systems and software engineering--Vocabulary[S] IEEE, 2017.
- [6] Yang Xiaoxing. Software defect prediction technology based on metric [D]. Hefei: University of Science and Technology of China, 2014.
- [7] McCabe T J. A complexity measure[J]. IEEE Transactions on Software Engineering, 1976,SE-2(4): 308-320.
- [8] Halstead M H. Elements of software science[M]. New York, USA: Elsevier, 1977.
- [9] Chidamber S R, Kemerer C F. A metrics suite for object oriented design[J].IEEE Transactions on Software Engineering, 1994, 20(6): 476-493.
- [10] Dash M,Liu H.Feature selection for classification[J].Intelligent Data Analysis,1997,1:131-156.
- [11] Jason Brownlee.An Introduction to Feature Selection. October 6, 2014 in Machine Learning Process. <https://machinelearningmastery.com/an-introduction-to-feature-selection/>
- [12] Huang Chen-jung, Yang Dian-xiu, Chuang Yi-ta. Application of wrapper approach and composite classifier to the stock trend prediction[J]. Expert Systems with Application,2008,4(34):2870-2878.
- [13] A.Günes Koru,H.Liu.Building effective defect prediction models in practice[J].IEEE Software,2005,22(6):23-29
- [14] D.W.Aha,D.Kibler,M.K.Albert.Instance-based learning algorithms.Machine Learning[J],1991, 6(1): 37-66
- [15] N.Bhatia.Survey of Nearest Neighbor Techniques.International Journal of Computer Science and Information Security[J], 2010, 8 (2): 302-305
- [16] Arvinder Kaur,Kamaldeep Kaur,Proceedings of the Second International Conference on Computer and Communication Technologies :Deepti Chopra.Application of Locally Weighted Regression for Predicting Faults Using Software Entropy Metrics[C]., Hyderabad, 24-26 July 2015.
- [17] Shiju Sathyadevan,Remya R. Nair. Comparative Analysis of Decision Tree Algorithms: ID3, C4.5 and Random Forest [J]. Computational Intelligence in Data Mining.2014,1(1),549-562.
- [18] Rizky Tri Asmono, Romi Satria Wahono and Abdul Syukur.Absolute Correlation Weighted Naïve Bayes for Software Defect Prediction [J]. Journal of Software Engineering.2015,1(1),38-45.
- [19] P.A. Selvaraj,Dr.P.Thangaraj.Support Vector Machine for Software Defect Prediction[J]. International Journal of Engineering & Technology Research.2013.1(2),68-76.
- [20] Gayathri M, A. Sudha. Software Defect Prediction System using Multilayer Perceptron Neural Network with Data Mining [J]. International Journal of Recent Technology and Engineering (IJRTE) .2014.3(2),54-59.
- [21] L. Breiman. Bagging predictors[J]. Machine Learning, 1996, 24(2): 123-140
- [22] Y. Freund, R.E. Schapir. Experiments with a new boosting algorithm. In: Proceedings of the 13th International Conference on Machine Learning, [C]Bari, Italy, 1996, 148-156
- [23] C.J.Merz.Using correspondence analysis to combine classifier[J].Machine Learning,1999,36(1-2): 33-58
- [24] Zhongbin Sun, Qinbao Song, Xiaoyan Zhu. Using Coding-Based Ensemble Learning to Improve Software Defect Prediction[J]. IEEE Transactions on Systems, 2012.42(6): 1806-1817.
- [25] López V, Fernández A, García S, et al. An insight into classification with imbalanced data:Empirical results and current trends on using data intrinsic characteristics[J].Information Sciences, 2013, 250: 113-141.
- [26] Chawla N V,Bowyer K W, Hall L O, et al. SMOTE: Synthetic minority over-sampling technique[J]. Journal of Artificial Intelligence Research, 2002, 16: 321-357.
- [27] Siers M J, Islam M Z. Software defect prediction using a cost sensitive decision forest and voting, and a potential solution to the class imbalance problem[J]. Information Systems, 2015,51: 62-71.
- [28] Guillem Colléll,DrazenPrelec,Kaustubh R.Patil.A simple plug-in bagging ensemble based on threshold-moving for classifying binary and multiclass imbalanced data [J]. Neurocomputing, 2018,275: 330-340.
- [29] Gray D, Bowes D, Davey N, et al. The misuse of the NASA metrics data program data sets for automated software defect prediction[C]. Proceedings of the 15th Annual Conference on Evaluation and Assessment in Software Engineering. London, England: IET, 2011: 96-103.
- [30] Menzies T, Greenwald J, Frank A. Data Mining Static Code Attributes to Learn Defect Predictors[J]. IEEE Transactions on Software Engineering. 2007, 33(1): 2-13.